

第19章 SysTick—系统定时器

本章参考资料《ARM Cortex™-M4F 技术参考手册》-4.5 章节 SysTick Timer(STK)，和 4.48 章节 SHPRx，其中 STK 这个章节有 SysTick 的简介和寄存器的详细描述。因为 SysTick 是属于 CM4 内核的外设，有关寄存器的定义和部分库函数都在 core_cm4.h 这个头文件中实现。所以学习 SysTick 的时候可以参考这两个资料，一个是文档，一个是源码。

19.1 SysTick 简介

SysTick—系统定时器是属于 CM4 内核中的一个外设，内嵌在 NVIC 中。系统定时器是一个 24bit 的向下递减的计数器，计数器每计数一次的时间为 1/SYSCLK，一般我们设置系统时钟 SYSCLK 等于 180M。当重载数值寄存器的值递减到 0 的时候，系统定时器就产生一次中断，以此循环往复。

因为 SysTick 是属于 CM4 内核的外设，所以所有基于 CM4 内核的单片机都具有这个系统定时器，使得软件在 CM4 单片机中可以很容易的移植。系统定时器一般用于操作系统，用于产生时基，维持操作系统的心跳。

19.2 SysTick 寄存器介绍

SysTick—系统定时有 4 个寄存器，简要介绍如下。在使用 SysTick 产生定时的时候，只需要配置前三个寄存器，最后一个校准寄存器不需要使用。

表 19-1 SysTick 寄存器汇总

寄存器名称	寄存器描述
CTRL	SysTick 控制及状态寄存器
LOAD	SysTick 重载数值寄存器
VAL	SysTick 当前数值寄存器
CALIB	SysTick 校准数值寄存器

表 19-2 SysTick 控制及状态寄存器

位段	名称	类型	复位值	描述
16	COUNTFLAG	R/W	0	如果在上次读取本寄存器后，SysTick 已经计到了 0，则该位为 1。
2	CLKSOURCE	R/W	0	时钟源选择位，0=AHB/8，1=处理器时钟 AHB
1	TICKINT	R/W	0	1=SysTick 倒数计数到 0 时产生 SysTick 异常请求，0=数到 0 时无动作。也可以通过读取 COUNTFLAG 标志位来确定计数器是否递减到 0
0	ENABLE	R/W	0	SysTick 定时器的使能位

表 19-3 SysTick 重载数值寄存器

位段	名称	类型	复位值	描述
23:0	RELOAD	R/W	0	当倒数计数至零时，将被重载的值

表 19-4 SysTick 当前数值寄存器

位段	名称	类型	复位值	描述
23:0	CURRENT	R/W	0	读取时返回当前倒计数的值，写它则使之清零，同时还会清除在 SysTick 控制及状态寄

				寄存器中的 COUNTFLAG 标志
--	--	--	--	--------------------

表 19-5 SysTick 校准数值寄存器

位段	名称	类型	复位值	描述
31	NOREF	R	0	NOREF flag. Reads as zero. Indicates that a separate reference clock is provided. The frequency of this clock is HCLK/8
30	SKEW	R	1	SKEW flag: Indicates whether the TENMS value is exact. Reads as one. Calibration value for the 1 ms inexact timing is not known because TENMS is not known. This can affect the suitability of SysTick as a software real time clock
23:0	TENMS	R	0	Calibration value. Indicates the calibration value when the SysTick counterruns on HCLK max/8 as external clock. The value is product dependent, please refer to the Product Reference Manual, SysTick Calibration Value section. When HCLK is programmed at the maximum frequency, the SysTick period is 1ms. If calibration information is not known, calculate the calibration value required from the frequency of the processor clock or external clock

系统定时器的校准数值寄存器在定时实验中不需要用到。有关各个位的描述这里引用手册里面的英文版本，比较晦涩难懂，暂时不知道这个寄存器用来干什么。有研究过的朋友可以交流，起个抛砖引玉的作用。

19.3 SysTick 定时实验

利用 SysTick 产生 1s 的时基，LED 以 1s 的频率闪烁。

19.3.1 硬件设计

SysTick 属于单片机内部的外设，不需要额外的硬件电路，剩下的只需一个 LED 灯即可。

19.3.2 软件设计

这里只讲解核心的部分代码，有些变量的设置，头文件的包含等并没有涉及到，完整的代码请参考本章配套的工程。我们创建了两个文件：bsp_SysTick.c 和 bsp_SysTick.h 文件用来存放 SysTick 驱动程序及相关宏定义，中断服务函数放在 stm32f4xx_it.h 文件中。

1. 编程要点

- 1、设置重装载寄存器的值
- 2、清除当前数值寄存器的值
- 3、配置控制与状态寄存器

2. 代码分析

SysTick 属于内核的外设，有关的寄存器定义和库函数都在内核相关的库文件 core_cm4.h 中。

SysTick 配置库函数

代码 19-1 SysTick 配置库函数

```
1  __STATIC_INLINE uint32_t SysTick_Config(uint32_t ticks)
2  {
3      // 不可能的重装载值，超出范围
4      if ((ticks - 1UL) > SysTick_LOAD_RELOAD_Msk) {
5          return (1UL);
6      }
7
8      // 设置重装载寄存器
9      SysTick->LOAD = (uint32_t)(ticks - 1UL);
10
11     // 设置中断优先级
12     NVIC_SetPriority (SysTick_IRQn, (1UL << __NVIC_PRIO_BITS) - 1UL);
13
14     // 设置当前数值寄存器
15     SysTick->VAL = 0UL;
16
17     // 设置系统定时器的时钟源为 AHBCLK=168M
18     // 使能系统定时器中断
19     // 使能定时器
20     SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk |
21                     SysTick_CTRL_TICKINT_Msk |
22                     SysTick_CTRL_ENABLE_Msk;
23     return (0UL);
24 }
```

用固件库编程的时候我们只需要调用库函数 SysTick_Config() 即可，形参 ticks 用来设置重装载寄存器的值，最大不能超过重装载寄存器的值 2^{24} ，当重装载寄存器的值递减到 0 的时候产生中断，然后重装载寄存器的值又重新装载往下递减计数，以此循环往复。紧随其后设置好中断优先级，最后配置系统定时器的时钟为 168M，使能定时器和定时器中断，这样系统定时器就配置好了，一个库函数搞定。

SysTick_Config() 库函数主要配置了 SysTick 中的三个寄存器：LOAD、VAL 和 CTRL，有关具体的部分看代码注释即可。其中还调用了固件库函数 NVIC_SetPriority() 来配置系统定时器的中断优先级，该库函数也在 core_cm4.h 中定义，原型如下：

```
1  __STATIC_INLINE void NVIC_SetPriority(IRQn_Type IRQn, uint32_t priority)
2  {
3      if ((int32_t)IRQn < 0) {
4          SCB->SHPR[(((uint32_t)(int32_t)IRQn) & 0xFUL)-4UL] =
5          (uint8_t)((priority << (8 - __NVIC_PRIO_BITS)) & (uint32_t)0xFFUL);
6      } else {
7          NVIC->IP[(((uint32_t)(int32_t)IRQn)] =
8          (uint8_t)((priority << (8 - __NVIC_PRIO_BITS)) & (uint32_t)0xFFUL);
9      }
10 }
```

因为 SysTick 属于内核外设，跟普通外设的中断优先级有些区别，并没有抢占优先级和子优先级的说法。在 STM32F407 中，内核外设的中断优先级由内核 SCB 这个外设的寄存器：SHPRx (x=1.2.3) 来配置。有关 SHPRx 寄存器的详细描述可参考《Cortex-M4 内核编程手册》4.4.8 章节。下面我们简单介绍下这个寄存器。

SPRH1-SPRH3 是一个 32 位的寄存器，但是只能通过字节访问，每 8 个字段控制着一个内核外设的中断优先级的配置。在 STM32F407 中，只有位 7:3 这四位有效，低四位没有用到，所以内核外设的中断优先级可编程为：0~15，只有 16 个可编程优先级，数值越小，优先级越高。如果软件优先级配置相同，那就根据他们在中断向量表里面的位置编号来决定优先级大小，编号越小，优先级越高。

表 19-6 系统异常优先级字段

异常	字段	寄存器描述
Memory management fault	PRI_4	SHPR1
Bus fault	PRI_5	
Usage fault	PRI_6	
SVCall	PRI_11	SHPR2
PendSV	PRI_14	SHPR3
SysTick	PRI_15	

如果要修改内核外设的优先级，只需要修改下面三个寄存器对应的某个字段即可。

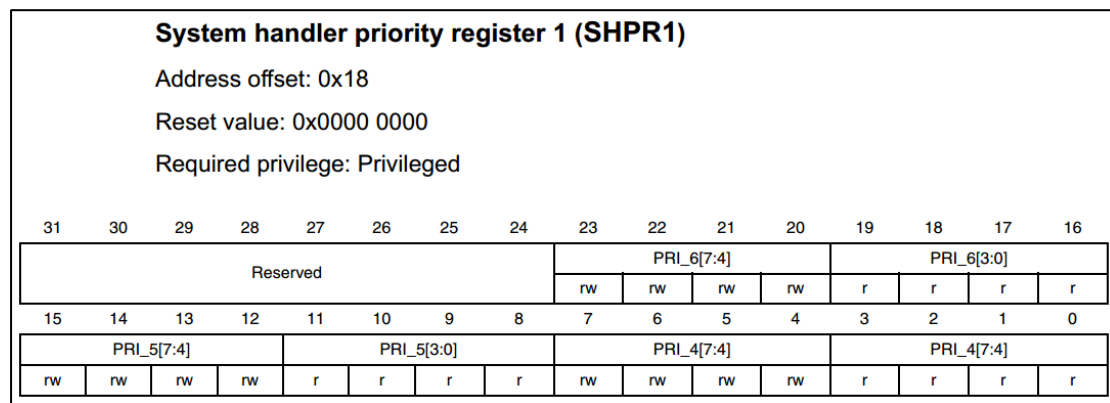


图 19-1 SHPR1 寄存器

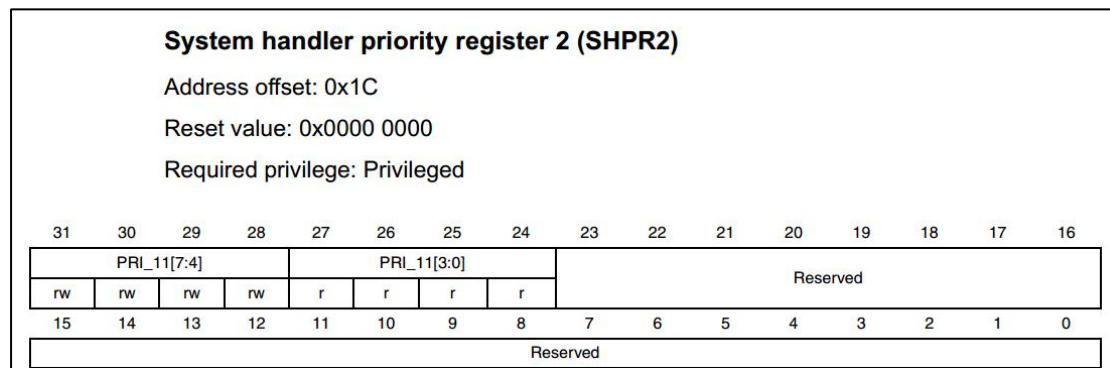


图 19-2 SHPR2 寄存器

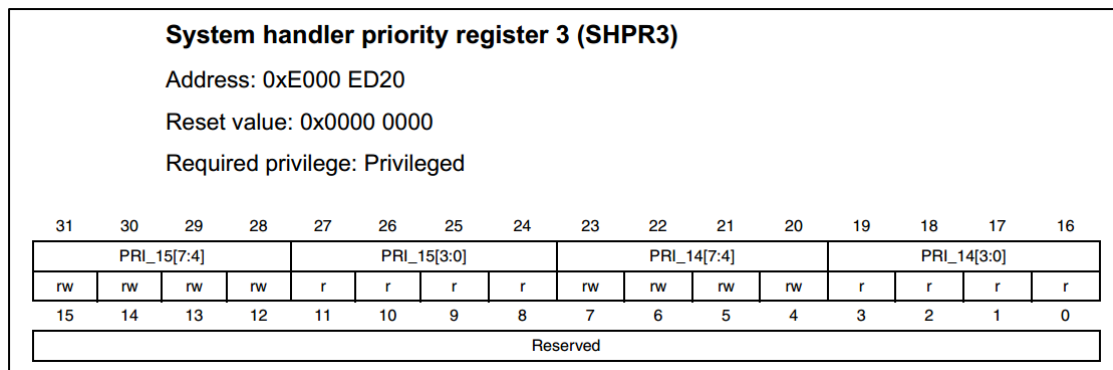


图 19-3 SHPR3 寄存器

在系统定时器中，配置优先级为 $(1UL \ll _NVIC_PRIO_BITS) - 1UL$ ，其中宏 $_NVIC_PRIO_BITS$ 为 4，那计算结果就等于 15，可以看出系统定时器此时设置的优先级在内核外设中是最低的。

```
1 // 设置系统定时器中断优先级
2 NVIC_SetPriority (SysTick_IRQn, (1UL << __NVIC_PRIO_BITS) - 1UL);
```

SysTick 初始化函数

代码 19-2 SysTick 初始化函数

```
1 /**
2  * @brief 启动系统滴答定时器 SysTick
3  * @param 无
4  * @retval 无
5  */
6 void SysTick_Init(void)
7 {
8     /* SystemFrequency / 1000    1ms 中断一次
9     * SystemFrequency / 100000    10us 中断一次
10    * SystemFrequency / 1000000    1us 中断一次
11    */
12    if (SysTick_Config(SystemCoreClock / 100000)) {
13        /* Capture error */
14        while (1);
15    }
16 }
```

SysTick 初始化函数由用户编写，里面调用了 SysTick_Config() 这个固件库函数，通过设置该固件库函数的形参，就决定了系统定时器经过多少时间就产生一次中断。

SysTick 中断时间的计算

SysTick 定时器的计数器是向下递减计数的，计数一次的时间 $T_{DEC} = 1/CLK_{AHB}$ ，当重载寄存器中的值 $VALUE_{LOAD}$ 减到 0 的时候，产生中断，可知中断一次的时间 $T_{INT} = VALUE_{LOAD} * T_{DEC}$ 中断 = $VALUE_{LOAD} / CLK_{AHB}$ ，其中 $CLK_{AHB} = 168MHz$ 。如果 $VALUE_{LOAD}$ 设置为 1680，那中断一次的时间 $T_{INT} = 1680 / 168M = 10us$ 。不过 1us 的中断没啥意义，整个程序的重心都花在进出中断上了，根本没有时间处理其他的任务。

```
SysTick_Config(SystemCoreClock / 100000)
```

SysTick_Config() 的形参我们配置为 $SystemCoreClock / 100000 = 168M / 100000 = 1680$ ，从刚刚分析我们知道这个形参的值最终是写到重载寄存器 LOAD 中的，从而可知我们现在把 SysTick 定时器中断一次的时间 $T_{INT} = 1680 / 168M = 10us$ 。

SysTick 定时时间的计算

当设置好中断时间 T_{INT} 后，我们可以设置一个变量 t ，用来记录进入中断的次数，那么变量 t 乘以中断的时间 T_{INT} 就可以计算出需要定时的时间。

SysTick 定时函数

现在我们定义一个微秒级别的延时函数，形参为 $nTime$ ，当用这个形参乘以中断时间 T_{INT} 就得出我们需要的延时时间，其中 T_{INT} 我们已经设置好为 $10\mu s$ 。关于这个函数的具体调用看注释即可。

```
1 /**
2  * @brief   us 延时程序, 10us 为一个单位
3  * @param
4  *   @arg nTime: Delay_us( 1 ) 则实现的延时为 1 * 10us = 10us
5  * @retval 无
6  */
7 void Delay_us( __IO u32 nTime)
8 {
9     TimingDelay = nTime;
10
11     while (TimingDelay != 0);
12 }
```

函数 `Delay_us()` 中我们等待 `TimingDelay` 为 0，当 `TimingDelay` 为 0 的时候表示延时时间到。变量 `TimingDelay` 在中断函数中递减，即 `SysTick` 每进一次中断即 $10\mu s$ 的时间 `TimingDelay` 递减一次。

SysTick 中断服务函数

```
1 void SysTick_Handler(void)
2 {
3     TimingDelay_Decrement();
4 }
```

中断复位函数调用了另外一个函数 `TimingDelay_Decrement()`，原型如下：

```
1 /**
2  * @brief   获取节拍程序
3  * @param   无
4  * @retval 无
5  * @attention 在 SysTick 中断函数 SysTick_Handler() 调用
6  */
7 void TimingDelay_Decrement(void)
8 {
9     if (TimingDelay != 0x00) {
10         TimingDelay--;
11     }
12 }
```

`TimingDelay` 的值等于延时函数中传进去的 $nTime$ 的值，比如 $nTime=100000$ ，则延时的时间等于 $100000*10\mu s=1s$ 。

主函数

```
1 int main(void)
2 {
3     /* LED 端口初始化 */
4     LED_GPIO_Config();
5
6     /* 配置 SysTick 为 10us 中断一次, 时间到后触发定时中断,
7      * 进入 stm32fxx_it.c 文件的 SysTick_Handler 处理, 通过数中断次数计时
8      */
9 }
```

```
9     SysTick_Init();
10
11     while (1) {
12
13         LED_RED;
14         Delay_us(100000);    // 10000 * 10us = 1000ms
15
16         LED_GREEN;
17         Delay_us(100000);    // 10000 * 10us = 1000ms
18
19         LED_BLUE;
20         Delay_us(100000);    // 10000 * 10us = 1000ms
21     }
22 }
```

主函数中初始化了 LED 和 SysTick，然后在一个 while 循环中以 1s 的频率让 LED 闪烁。

另外一种更简洁的定时编程

上面的实验，我们是使用了中断，而且经过多个函数的调用，还使用了全局变量，理解起来挺费劲的，其实还有另外一种更简洁的写法。我们知道，systick 的 counter 从 reload 值往下递减到 0 的时候，CTRL 寄存器的位 16:countflag 会置 1，且读取该位的值可清 0，所有我们可以使用软件查询的方法来实现延时。具体代码见代码 19-3 和代码 19-4，我敢肯定这样的写法，初学者肯定会更喜欢，因为它直接，套路浅。

代码 19-3 systick 微秒级延时

```
1 void SysTick_Delay_Us( __IO uint32_t us)
2 {
3     uint32_t i;
4     SysTick_Config(SystemCoreClock/1000000);
5
6     for (i=0; i<us; i++) {
7         // 当计数器的值减小到 0 的时候，CTRL 寄存器的位 16 会置 1
8         while ( !((SysTick->CTRL)&(1<<16)) );
9     }
10    // 关闭 SysTick 定时器
11    SysTick->CTRL &=~SysTick_CTRL_ENABLE_Msk;
12 }
```

代码 19-4 systick 毫秒级延时

```
1 void SysTick_Delay_Ms( __IO uint32_t ms)
2 {
3     uint32_t i;
4     SysTick_Config(SystemCoreClock/1000);
5
6     for (i=0; i<ms; i++) {
7         // 当计数器的值减小到 0 的时候，CTRL 寄存器的位 16 会置 1
8         // 当置 1 时，读取该位会清 0
9         while ( !((SysTick->CTRL)&(1<<16)) );
10    }
11    // 关闭 SysTick 定时器
12    SysTick->CTRL &=~ SysTick_CTRL_ENABLE_Msk;
13 }
```

在这两个微秒和毫秒级别的延时函数中，我们还是调用了 SysTick_Config 这个固件库函数，有关这个函数的说明具体见代码 19-5。配套代码注释理解即可。其中 SystemCoreClock 是一个宏，大小为 168000000，如果不想使用这个宏，也可以直接改成数字。

代码 19-5 systick 配置函数


```
1 // 这个 固件库函数 在 core_cm3.h 中
2 static __INLINE uint32_t SysTick_Config(uint32_t ticks)
3 {
4     // reload 寄存器为 24bit, 最大值为 2^24
5     if (ticks > SysTick_LOAD_RELOAD_Msk) return (1);
6
7     // 配置 reload 寄存器的初始值
8     SysTick->LOAD = (ticks & SysTick_LOAD_RELOAD_Msk) - 1;
9
10    // 配置中断优先级为 1<<4 -1 = 15, 优先级为最低
11    NVIC_SetPriority (SysTick_IRQn, (1<<__NVIC_PRIO_BITS) - 1);
12
13    // 配置 counter 计数器的值
14    SysTick->VAL = 0;
15
16    // 配置 systick 的时钟为 72M
17    // 使能中断
18    // 使能 systick
19    SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk |
20                   SysTick_CTRL_TICKINT_Msk |
21                   SysTick_CTRL_ENABLE_Msk;
22    return (0);
23 }
```
